Практическая работа 3

1. Базовые тесты

Базовый тест 1

```python
@pytest.mark.basic
def test_register_user(auth):
    auth.register_user("user1", "password123", "CountryA", 1000)
    user = auth.authenticate_user("user1", "password123")
    assert user is not None
    assert user[1] == "user1"
    assert user[3] == "CountryA"
```

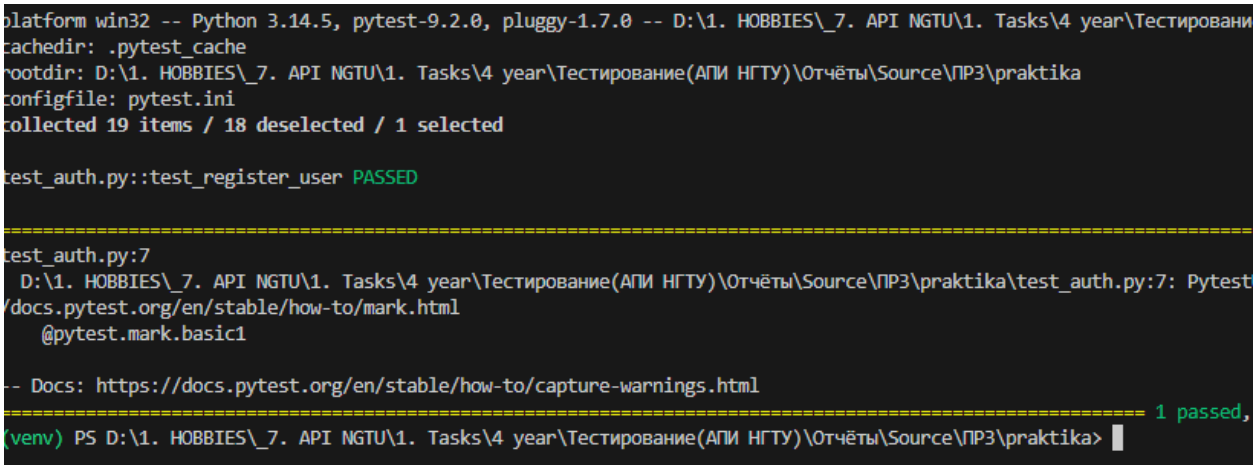


Рисунок 1 – Базовый тест 1 пройден

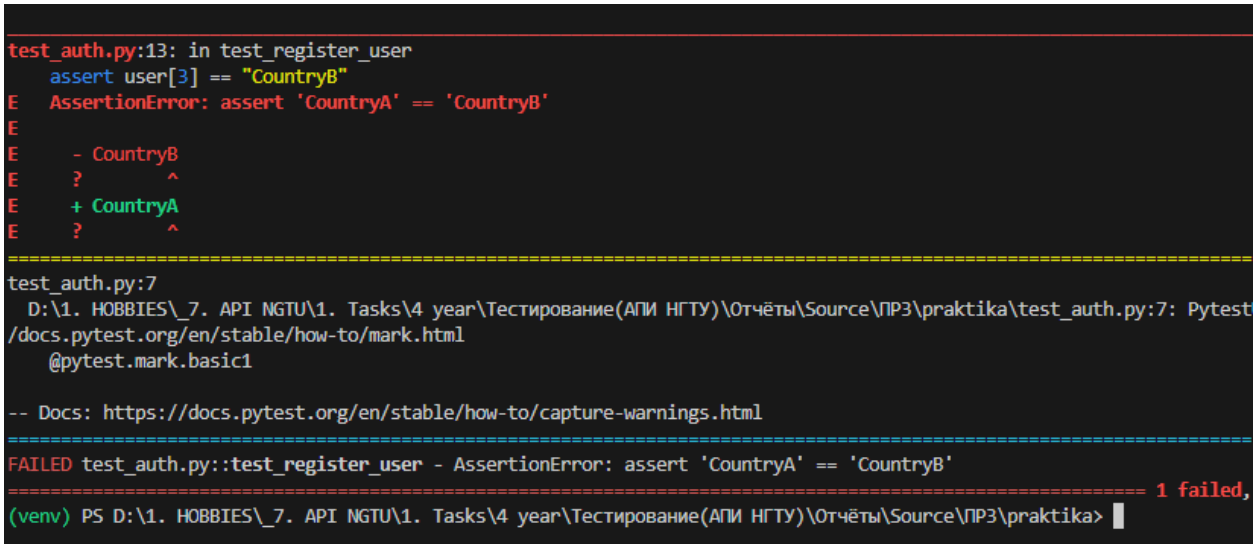


Рисунок 2 – Базовый тест 1 провален

Базовый тест 2

```python
@pytest.mark.basic
def test_authenticate_wrong_password(auth):
    auth.register_user("user1", "password123", "CountryA", 1000)
    assert auth.authenticate_user("user1", "wrongpass") is None
```

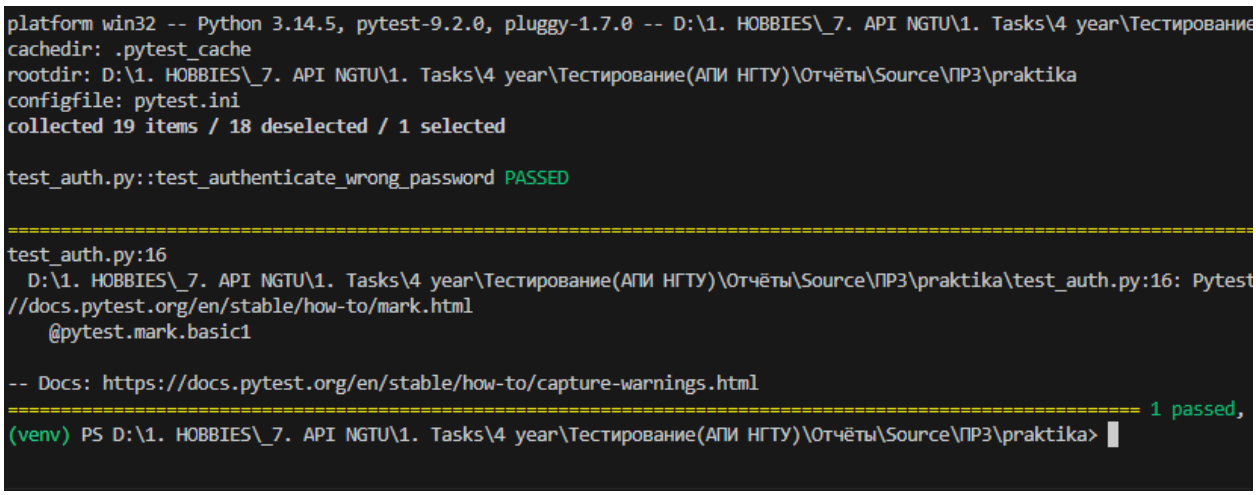


Рисунок 3 – Базовый тест 2 пройден

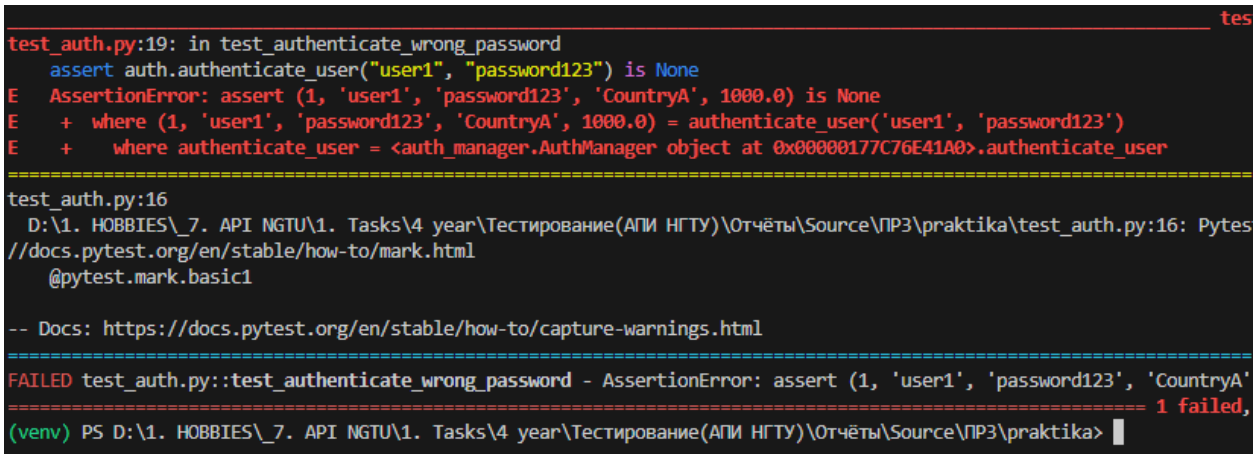


Рисунок 4 – Базовый тест 2 провален

Базовый тест 3

```python
@pytest.mark.basic
def test_delete_user(auth):
    auth.register_user("user1", "password123", "CountryA", 1000)
    user = auth.authenticate_user("user1", "password123")
    auth.delete_user(user[0])
    assert auth.authenticate_user("user1", "password123") is None
```

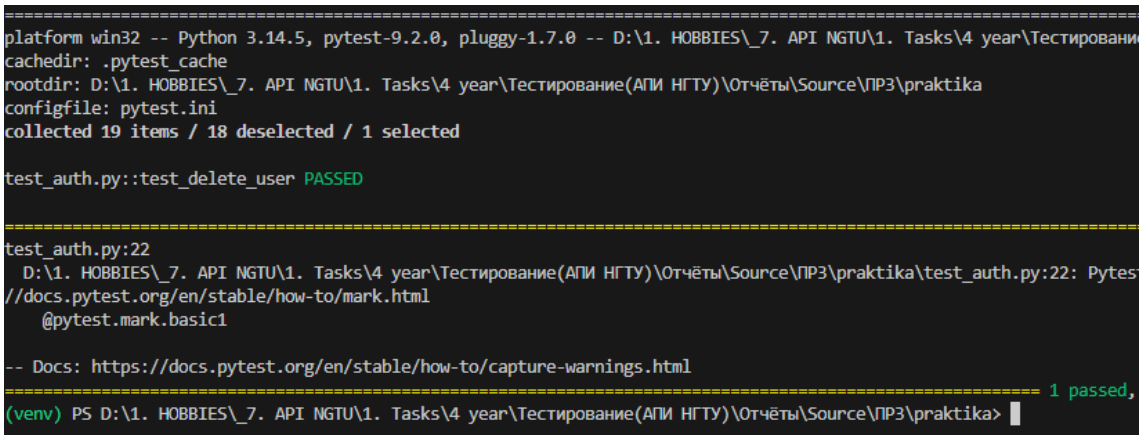


Рисунок 5 – Базовый тест 3 пройден

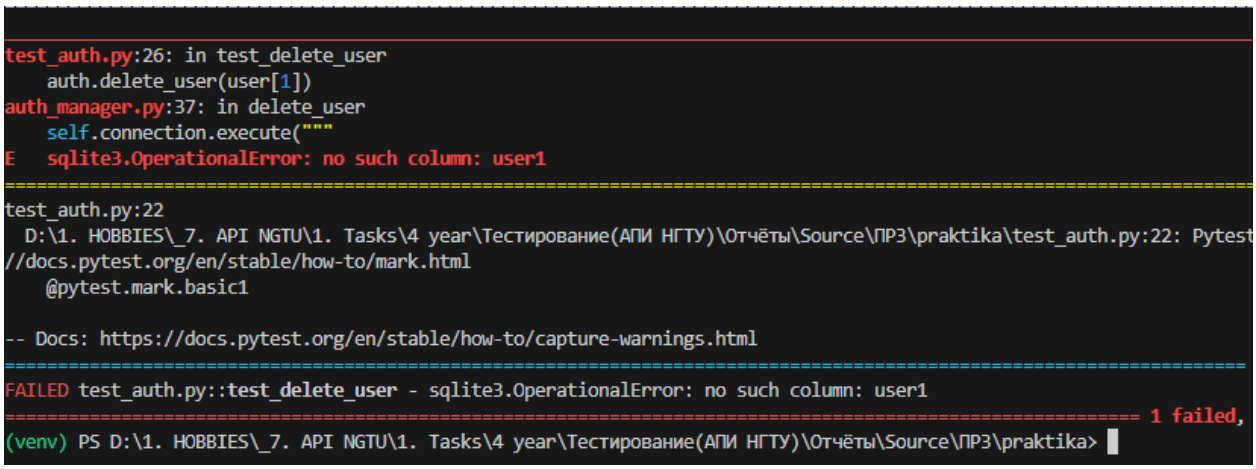


Рисунок 6 – Базовый тест 3 провален

2. Параметризированные тесты

Параметризированный тест 1

```
@pytest.mark.parametrized
@pytest.mark.parametrize("country, expected_count", [
    ("CountryA", 2),
    ("CountryB", 1),
    ("CountryC", 0),
])
def test_count_users_by_country(filled_auth, country, expected_count):
    assert filled_auth.count_users_by_country(country) == expected_count
```

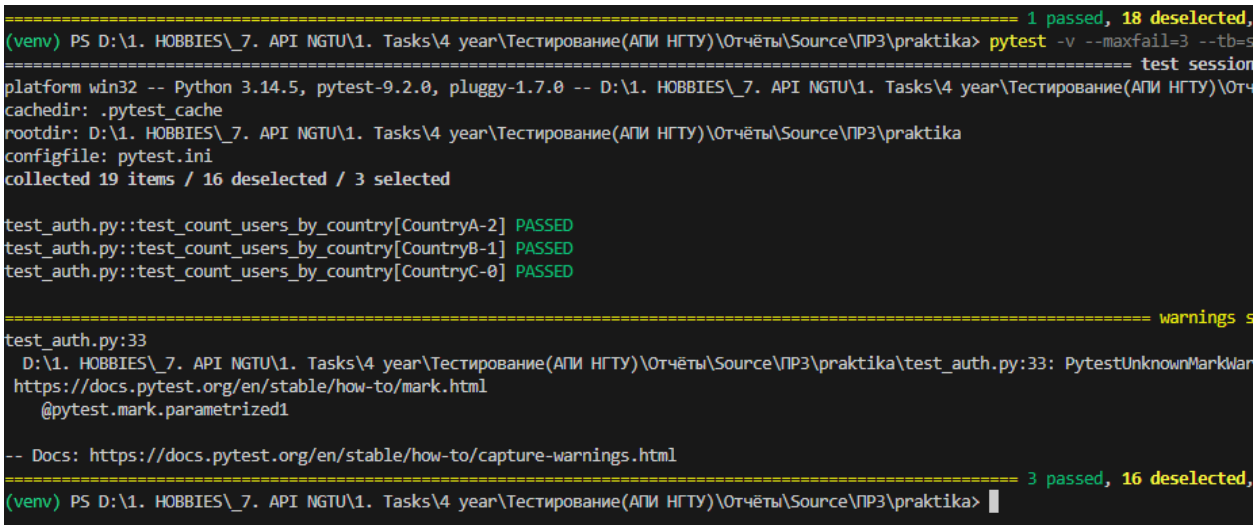


Рисунок 7 – Параметризированный тест 1 пройден

```
E    +  where 1 = count_users_by_country('CountryB')
E    +    where count_users_by_country = <auth_manager.AuthManager object at 0x00000252E5761090>.count_users_by_countr
_____________________________________________________________________________________________ test_cou
test_auth.py:40: in test_count_users_by_country
    assert filled_auth.count_users_by_country(country) == expected_count
E   AssertionError: assert 0 == 1
E    +  where 0 = count_users_by_country('CountryC')
E    +    where count_users_by_country = <auth_manager.AuthManager object at 0x00000252E5761BD0>.count_users_by_countr
=============================================================================================================
test_auth.py:33
  D:\1. HOBBIES\_7. API NGTU\1. Tasks\4 year\Тестирование(АПИ НГТУ)\Отчёты\Source\ПР3\praktika\test_auth.py:33: Pytest
 https://docs.pytest.org/en/stable/how-to/mark.html
    @pytest.mark.parametrized1

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
=============================================================================================================
FAILED test_auth.py::test_count_users_by_country[CountryA-1] - AssertionError: assert 2 == 1
FAILED test_auth.py::test_count_users_by_country[CountryB-0] - AssertionError: assert 1 == 0
FAILED test_auth.py::test_count_users_by_country[CountryC-1] - AssertionError: assert 0 == 1
!!!
```

```
E     + CountryB
_____ test_register_
test_auth.py:53: in test_register_multiple_users
    assert user[3] == password
E   AssertionError: assert 'CountryC' == 'p3'
E
E     - p3
E     + CountryC
test_auth.py:43
  D:\1. HOBBIES\_7. API NGTU\1. Tasks\4 year\Тестирование(АПИ НГТУ)\Отчёты\Source\ПР3\praktika\test_auth.py:43: Pytest
 https://docs.pytest.org/en/stable/how-to/mark.html
    @pytest.mark.parametrized1

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
FAILED test_auth.py::test_register_multiple_users[u1-p1-CountryA-100] - AssertionError: assert 'CountryA' == 'p1'
FAILED test_auth.py::test_register_multiple_users[u2-p2-CountryB-0] - AssertionError: assert 'CountryB' == 'p2'
FAILED test_auth.py::test_register_multiple_users[u3-p3-CountryC-9999.99] - AssertionError: assert 'CountryC' == 'p3'
!!!
```

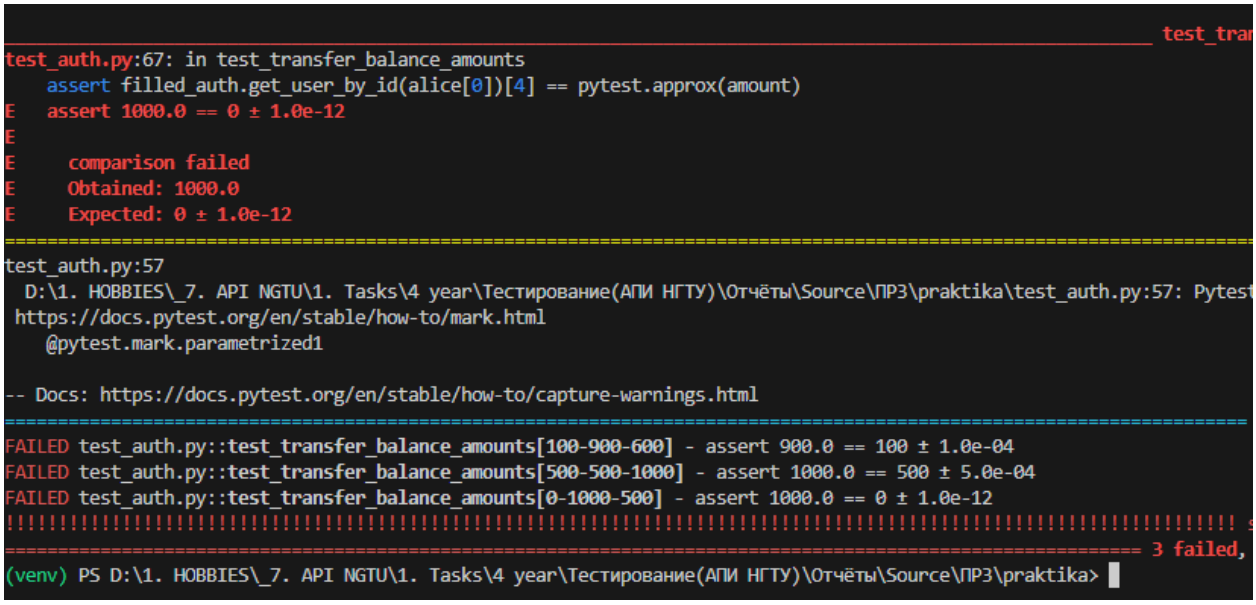


Рисунок 12 – Параметризированный тест 3 провален

3. Тесты исключений

Тест исключения 1

```python
@pytest.mark.exception
def test_transfer_insufficient_funds(filled_auth):
    alice = filled_auth.authenticate_user("alice", "pass1")
    bob = filled_auth.authenticate_user("bob", "pass2")
    with pytest.raises(ValueError, match="Insufficient funds"):
        filled_auth.transfer_balance(bob[0], alice[0], 10_000)
```

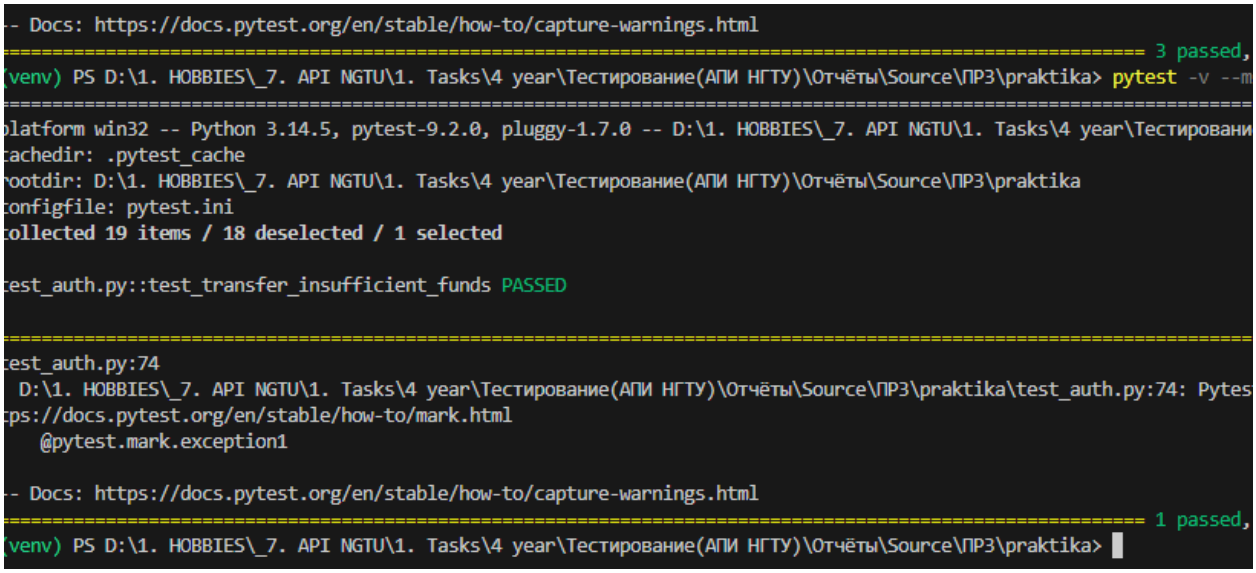


Рисунок 13 – Тест исключения 1 пройден

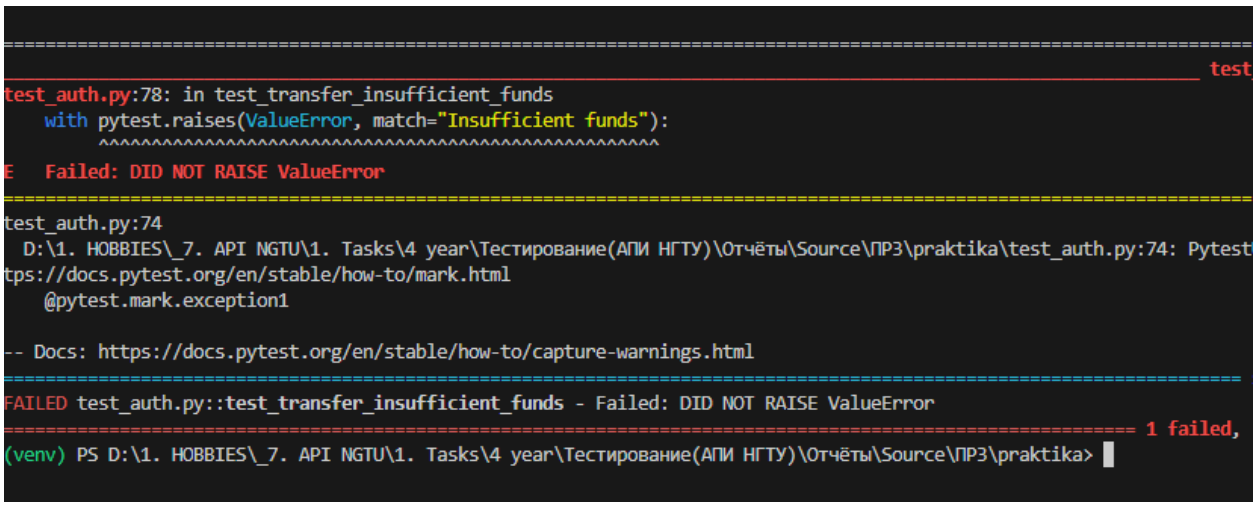


Рисунок 14 – Тест исключения 1 провален

Тест исключения 2

```
@pytest.mark.exception
def test_register_duplicate_username(filled_auth):
    import sqlite3
    with pytest.raises(sqlite3.IntegrityError):
        filled_auth.register_user("alice", "another", "CountryA", 10)
```

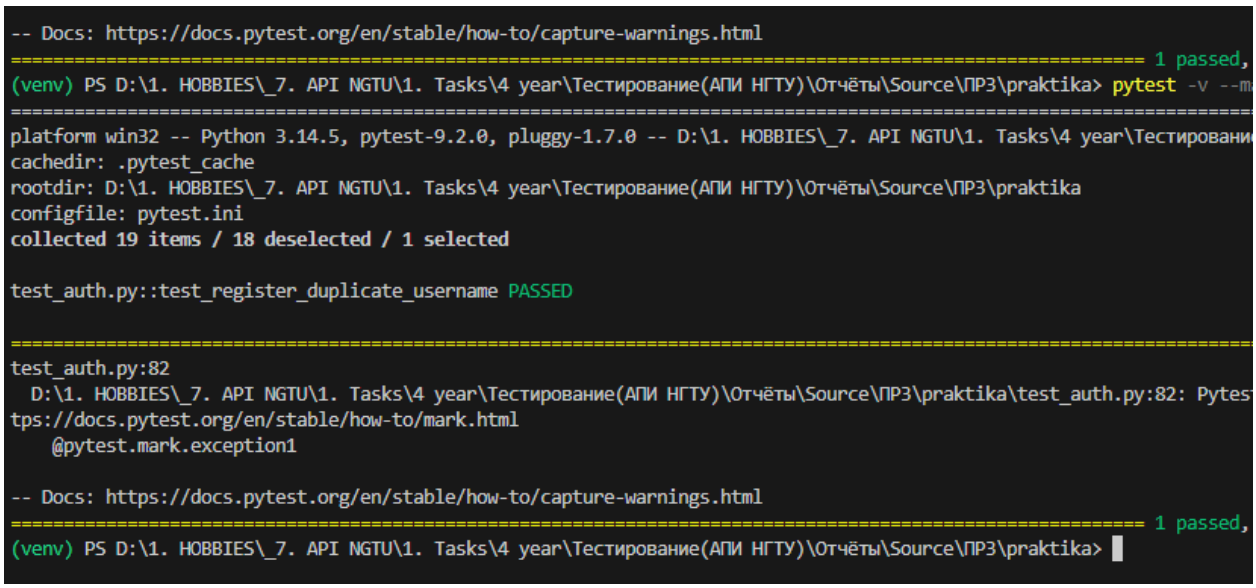


Рисунок 15 – Тест исключения 2 пройден

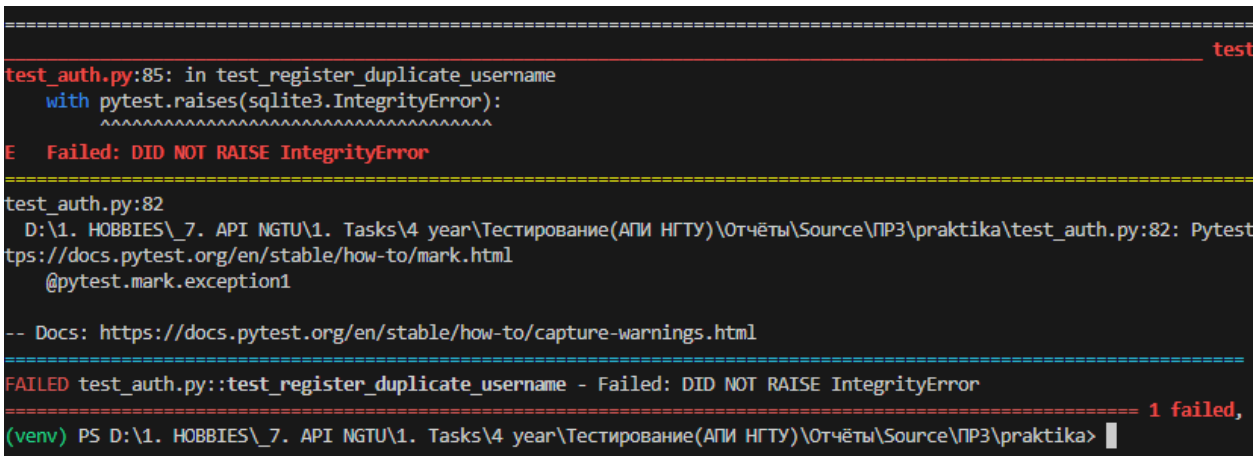


Рисунок 16 – Тест исключения 2 провален

4. Тесты на SQL-инъекции

Тест на SQL-инъекцию 1

```python
@pytest.mark.security1
def test_sql_injection_in_authenticate(filled_auth):
    """
        Классическая инъекция: ' OR '1'='1
        При параметризованных запросах вернёт None (пользователь не найден).
        При уязвимом .format() вернёт первого пользователя в таблице.
    """
    result = filled_auth.authenticate_user("' OR '1'='1' --", "anything")
    assert result is None, "SQL-инъекция прошла — запрос уязвим!"
```

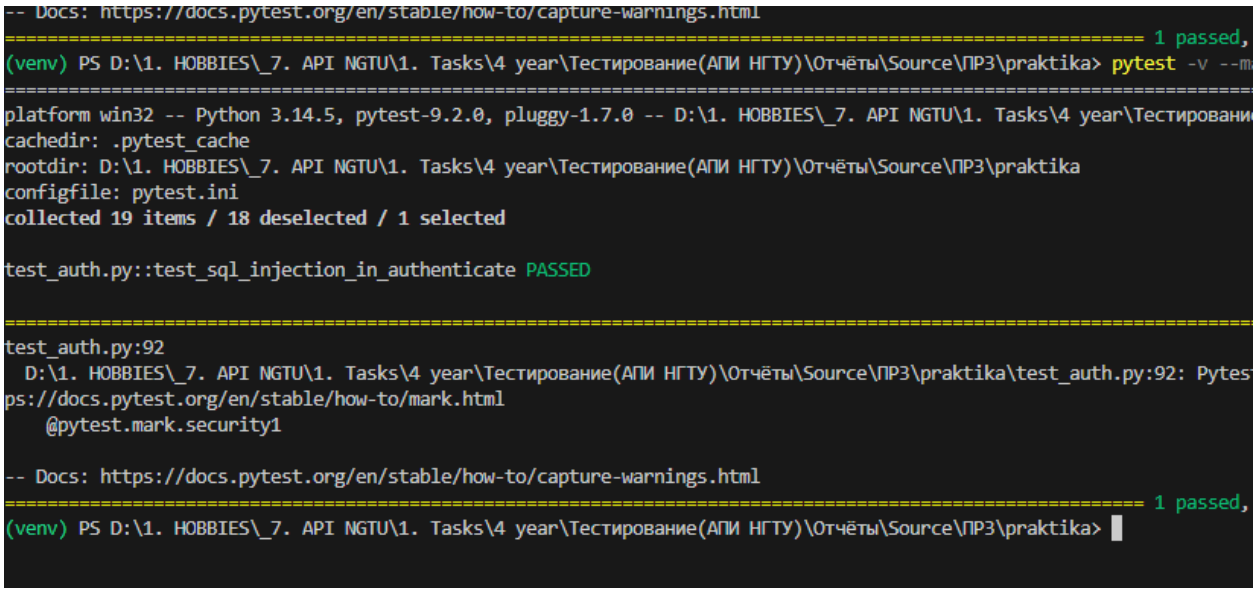


Рисунок 17 – Тест на SQL-инъекцию 1 пройден

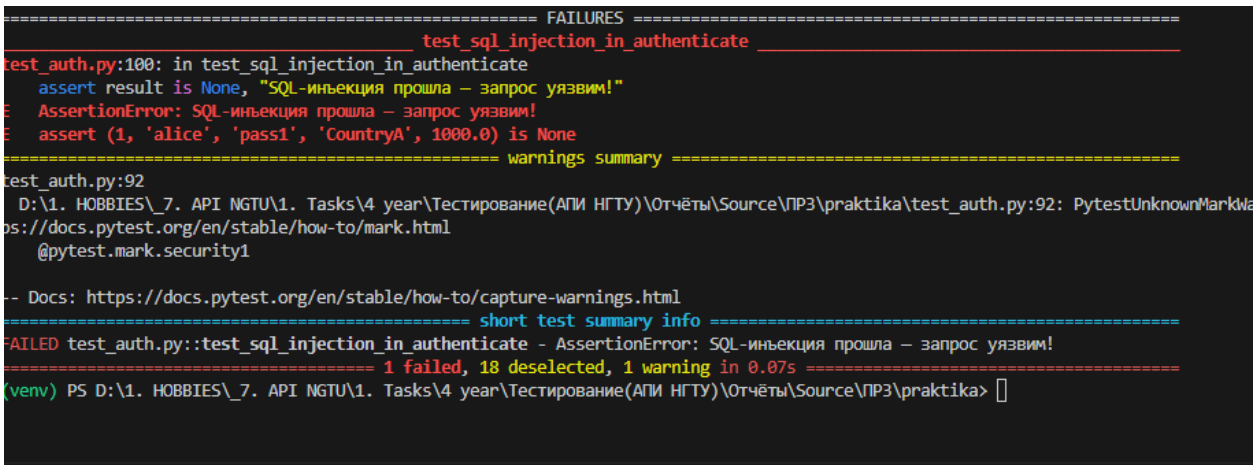


Рисунок 18 – Тест на SQL-инъекцию 1 провален

Тест на SQL-инъекцию 2

```python
@pytest.mark.security1
def test_sql_injection_in_count(filled_auth):
    """
        Инъекция в count_users_by_country: ' OR '1'='1
        Параметризованный запрос вернёт 0 для несуществующей страны.
    """
    count = filled_auth.count_users_by_country("' OR '1'='1")
    assert count == 0, "SQL-инъекция в count прошла — запрос уязвим!"
```

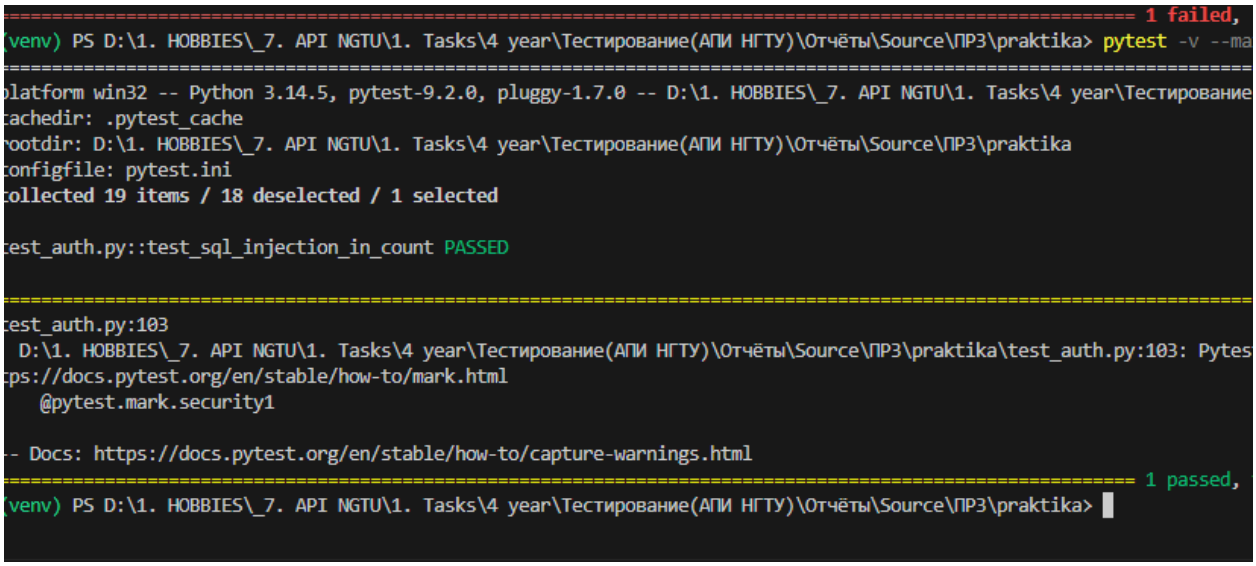


Рисунок 19 – Тест на SQL-инъекцию 2 пройден

```
test_auth.py:110: in test_sql_injection_in_count
    assert count == 0, "SQL-инъекция в count прошла — запрос уязвим!"
E   AssertionError: SQL-инъекция в count прошла — запрос уязвим!
E   assert 3 == 0
test_auth.py:103
  D:\1. HOBBIES\_7. API NGTU\1. Tasks\4 year\Тестирование(АПИ НГТУ)\Отчёты\Source\ПР3\praktika\test_auth.py:103: Pytes
tps://docs.pytest.org/en/stable/how-to/mark.html
    @pytest.mark.security1

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
FAILED test_auth.py::test_sql_injection_in_count - AssertionError: SQL-инъекция в count прошла — запрос уязвим!
1 failed,
(venv) PS D:\1. HOBBIES\_7. API NGTU\1. Tasks\4 year\Тестирование(АПИ НГТУ)\Отчёты\Source\ПР3\praktika>
```

Рисунок 20 – Тест на SQL-инъекцию 2 провален

5. Фикстуры БД

Фикстура БД 1

```python
@pytest.mark.db
def test_db_fixture_is_isolated(auth, db):
    """Каждый тест получает свежую in-memory БД."""
    auth.register_user("x", "y", "CountryA", 1)
    cur = db.cursor()
    cur.execute("SELECT COUNT(*) FROM users")
    assert cur.fetchone()[0] == 1
```

```
atform win32 -- Python 3.14.5, pytest-9.2.0, pluggy-1.7.0 -- D:\1. HOBBIES\_7. API NGTU\1. Tasks\4 year\Тестирование
chedir: .pytest_cache
otdir: D:\1. HOBBIES\_7. API NGTU\1. Tasks\4 year\Тестирование(АПИ НГТУ)\Отчёты\Source\ПР3\praktika
nfigfile: pytest.ini
llected 19 items / 18 deselected / 1 selected

st_auth.py::test_db_fixture_is_isolated PASSED

st_auth.py:103
D:\1. HOBBIES\_7. API NGTU\1. Tasks\4 year\Тестирование(АПИ НГТУ)\Отчёты\Source\ПР3\praktika\test_auth.py:103: Pytes
s://docs.pytest.org/en/stable/how-to/mark.html
  @pytest.mark.security1

 Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
1 passed,
env) PS D:\1. HOBBIES\_7. API NGTU\1. Tasks\4 year\Тестирование(АПИ НГТУ)\Отчёты\Source\ПР3\praktika>
```

Рисунок 21 – Тест на фикстуру бд 1 пройден

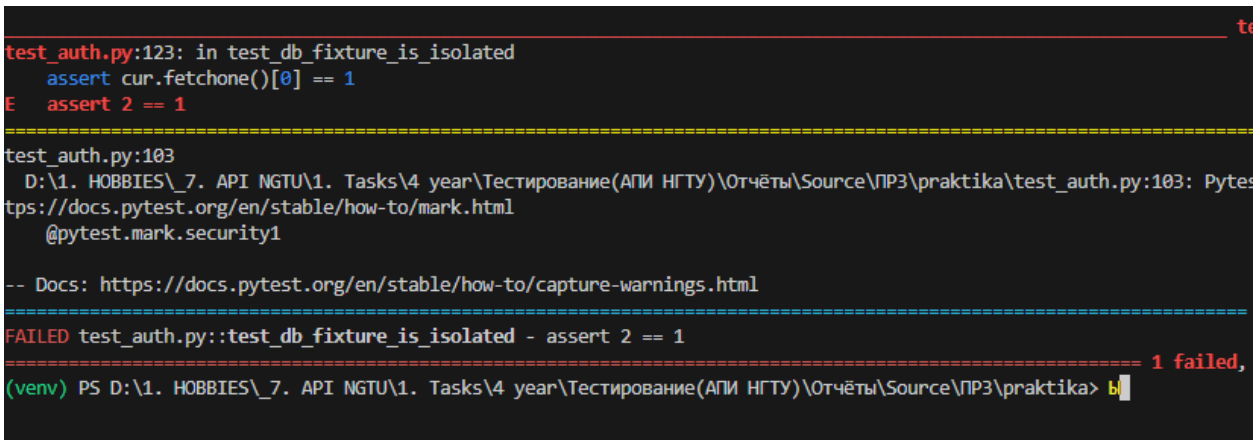


Рисунок 22 – Тест на фикстуру бд 1 провален

6. Пропускаемые тесты

Пропускаемый тест 1

```
@pytest.mark.basic
def test_python_version():
    assert True
```

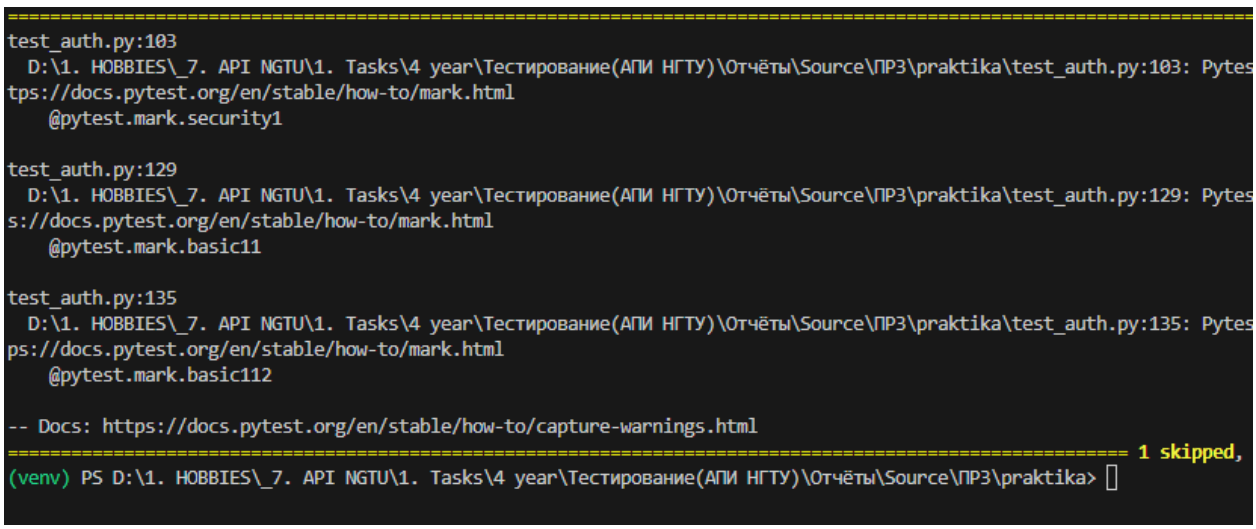


Рисунок 23 Пропускаемый тест 1 пропущен

Пропускаемый тест 2

```
@pytest.mark.basic11
def test_skipped_demo():
    assert False  # специально ложное, но скипнутое
```

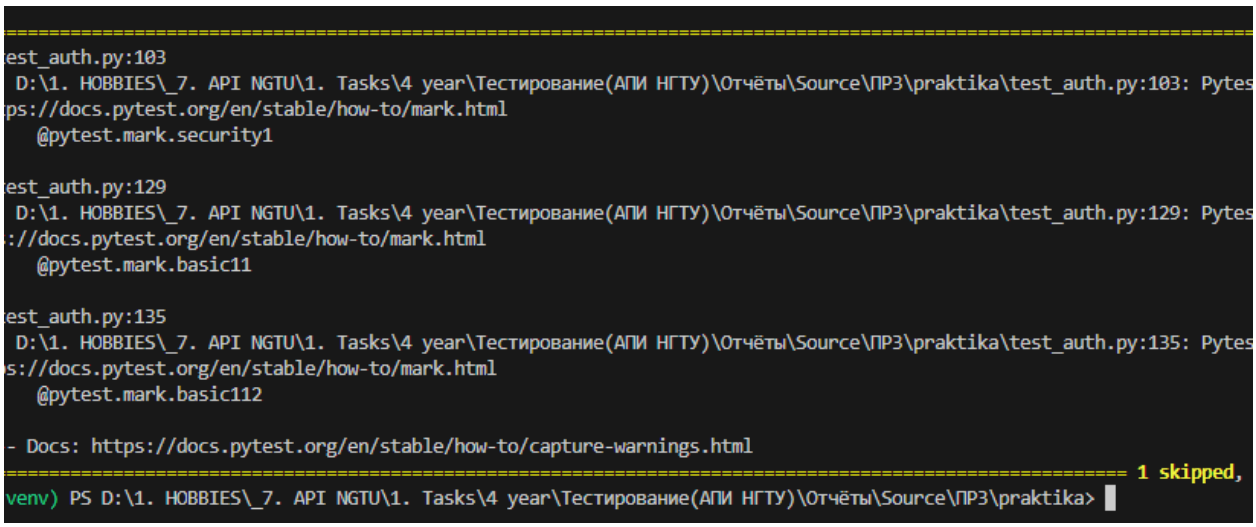


Рисунок 24 Пропускаемый тест 2 пропущен